Table des matières

Exercices corrigés 04 — Extraire d’une base ou d’une API 1
Questions courtes 1
Questions pratiques 2
Diagnostics 2
Questions à développer 2
Mises en situation 3
Corrigé 4

Exercices corrigés 04 — Extraire d’une base ou d’une API

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les questions courtes vérifient que le cours est clair dans votre tête ; les questions pratiques se font sur la base de l’ERP démarrée et sur la `RAW` du Cours 03. Tout vient de la Théorie 04 et du Labo 04.

Le corrigé suit les énoncés, en fin de document. Répondez à l’énoncé en entier avant d’y passer : c’est en butant sur une question qu’on découvre ce qu’on avait lu sans le comprendre.

Questions courtes

1. Tirer ou recevoir. Qu’est-ce qui distingue une source qui pousse d’une source qu’on tire, et quelle responsabilité chacune laisse au pipeline ?
2. ODBC. Nommez les quatre pièces qui se passent une requête, de votre script jusqu’à la base.
3. Le pilote comme contrat. Qu’est-ce qui change dans votre code si la base passe de PostgreSQL à SQL Server ?
4. Lire sans nuire. Donnez trois précautions à prendre avant d’extraire d’une base de production.
5. Le filigrane. Qu’est-ce qu’un filigrane, et pourquoi `AEDAT` seul ne suffit-il pas chez Boréal ?
6. Les angles morts. Citez deux choses qu’une capture par filigrane ne voit pas, et ce qu’on fait pour chacune.
7. Les documents de modification. Que gardent `CDHDR` et `CDPOS` que la table `KNA1` ne garde pas ?
8. Le journal de la base. Qu’apporte la capture par journal que les deux autres méthodes n’ont pas, et que demande-t-elle en retour ?

© 2026 iAlexMG. Tous droits réservés.

9. La pagination. Dites ce que valent `numberMatched` et `numberReturned` dans une réponse de l'API d'ECCC, et comment on sait qu'il reste des pages.
10. Réessayer ou pas. Pour chacun de ces codes, dites s'il faut réessayer : 400, 429, 500, et une coupure du réseau sans code.
11. La temporisation exponentielle. Expliquez le principe, et dites pourquoi on borne le nombre d'essais.

Questions pratiques

Ici on écrit vraiment : des requêtes, des commandes. Tapez votre réponse avant de la comparer. Les requêtes DuckDB se lancent dans l'interface web, comme à l'étape 3 du labo, après `ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);`, les tables nommées en entier (`entrepot.raw.…`) ; celles de la base, dans `psql`, ouvert dans le conteneur depuis la fenêtre Ubuntu, comme à l'étape 1 ; le script de l'API, depuis `Projet_Boreal_intermediaire`, par son chemin.

P1. Le filigrane de juillet 2024. Écrivez la requête qui compte, dans l'état reconstruit de `VBAK`, les commandes créées ou modifiées en juillet 2024, puis celle qui compte les lignes du lot `2024-07`. Les deux nombres doivent concorder. Une requête par cellule.

P2. Une requête sur la base. Dans `psql`, écrivez la requête qui donne, pour chaque concessionnaire fermé (`LOEVM` non vide), son numéro, son nom et sa province. Combien y en a-t-il ?

P3. Les paliers révisés. Dans `psql`, écrivez la requête qui compte les concessionnaires par ancien et par nouveau palier, pour la révision du 2024-07-01 : le champ `KUKLA` de `sap.cdpos`, daté par `sap.cdhdr`. Combien sont montés d'un palier, combien sont descendus ? Le dictionnaire de Boréal donne le sens des codes.

P4. La taille de page. Dans `extraire_meteo_api.py`, passez `PAGE` de 50 à 100, relancez le script, puis remettez 50. Combien de pages le script lit-il, et le fichier du bronze change-t-il ? Pourquoi ne pas toujours demander la plus grande page que l'API accepte ?

Diagnostics

On vous montre quelque chose qui ne va pas. Dites ce qui cloche, pourquoi, et donnez le correctif. Ces situations sont toutes arrivées à quelqu'un.

E1. Un pipeline extrait chaque nuit les commandes modifiées depuis la veille, avec le filtre `aedat >= <filigrane>`. Les rapports mensuels tombent juste, mais le tableau de bord des nouvelles commandes reste désespérément vide.

E2. Un script d'extraction ODBC lit une table de 40 millions de lignes avec `fetchall()`, puis écrit le résultat dans un CSV. Il tourne sur le poste d'un collègue, où il finit par échouer avec une erreur de mémoire ; sur le serveur, il fait tomber les temps de réponse de l'ERP pendant vingt minutes.

Questions à développer

Ces questions ne se répondent pas en un mot : on veut un court paragraphe de raisonnement. Rédigez votre réponse avant d'ouvrir le corrigé.

© 2026 iAlexMG. Tous droits réservés.

A. Choisir une capture de changements. Pour chacune de ces trois sources, dites quelle méthode de capture vous retiendriez et pourquoi : les commandes de l'ERP, les fiches des concessionnaires, et une table de stocks mise à jour toutes les minutes par un automate, où les lignes se suppriment.

B. Le coût d'une extraction. Un collègue propose de remplacer les extraits mensuels de l'ERP par une extraction ODBC complète chaque nuit, « pour avoir toujours l'état courant ». Évaluez la proposition : ce qu'elle apporte, ce qu'elle coûte, ce qu'elle perd.

Mises en situation

Le format le plus proche du travail réel. On vous décrit un organisme, ses données et ses contraintes ; à vous de décider et de justifier. Il n'y a pas une seule bonne réponse, mais il y a des réponses indéfendables. Le corrigé donne une solution défendable et dit ce qui serait acceptable aussi.

M1. Portail des permis de la Ville de Rivière-Blanche. La ville publie une API REST des permis émis, paginée par curseur (chaque réponse porte un jeton `suivant`), plafonnée à 500 permis par page et limitée à 1 000 requêtes par heure, avec une réponse 429 au-delà. L'historique compte 900 000 permis, et 300 s'ajoutent chaque jour.

Décrivez l'extraction initiale, puis l'extraction quotidienne : ce que vous gardez entre deux exécutions, comment vous respectez la limite, et comment vous prouvez qu'il ne manque rien.

M2. (avancé) Assurances Val-Perdu. Leur base de polices, dans Oracle, pèse 400 Go, dont une table de mouvements de 2 milliards de lignes. La fenêtre d'extraction nocturne est de deux heures, les polices résiliées sont supprimées de la table des polices, et l'équipe d'exploitation refuse tout accès au journal de transactions « pour des raisons de sécurité ».

Proposez une stratégie d'ingestion complète : méthode de capture, fréquence, ce que vous demandez à l'équipe de l'ERP, et ce que vous faites des suppressions. Dites ce qui vous ferait changer d'avis.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés. Les chiffres sont ceux de la génération par défaut de Boréal, chargée par `charger_raw.py` ; les requêtes se lancent depuis `Projet_Boreal_intermediaire`.

Questions courtes

1. Une source qui pousse livre ce qu'elle a, quand elle l'a : il faut être là pour recevoir, et accepter les doublons que la livraison « au moins une fois » produit. Une source qu'on tire attend la demande : c'est le pipeline qui sait quoi demander, quand, et qui doit parcourir toutes les pages ou toutes les lignes sans rien manquer.

Théorie : `Theorie_04_DE.md`, section 1.

2. L'application, qui ouvre la connexion et envoie le SQL (`extraire_erp_odbc.py`, avec `pyodbc`) ; le gestionnaire de pilotes, celui de Windows, qui trouve le pilote demandé ; le pilote, psqlODBC, qui traduit les appels dans le protocole de la base ; la base elle-même, PostgreSQL 18, dans son conteneur, sur le port 5433.

Théorie : `Theorie_04_DE.md`, section 2.

3. Presque rien : la chaîne de connexion nomme un autre pilote, et le dialecte SQL change parfois. Les appels restent les mêmes, parce que `pyodbc` suit l'interface standard des bases de données de Python (DB-API 2.0). C'est tout l'intérêt du pilote comme contrat.

Théorie : `Theorie_04_DE.md`, section 2.

4. Extraire hors des heures de pointe, ou sur une réplique en lecture. Se connecter avec un compte qui n'a que le droit de lire. Lire par paquets (`fetchmany`), et ne demander que les colonnes et les lignes utiles, plutôt que de balayer la table entière.

Théorie : `Theorie_04_DE.md`, section 2.

5. Le filigrane est la plus haute date de modification déjà extraite : on demande la suite, pas tout. `AEDAT` seul ne suffit pas chez Boréal parce qu'il vaut `00000000` tant que la commande n'a jamais été modifiée. En décembre 2025, un filigrane sur `AEDAT` seul rend 37 commandes et en rate 390 ; il faut `ERDAT >= f OR AEDAT >= f` pour retrouver les 427 de l'extrait.

Théorie : `Theorie_04_DE.md`, section 3.

6. Les suppressions : une ligne disparue ne revient dans aucune requête ; la parade est une suppression logique, comme l'indicateur `LOEVM` des concessionnaires fermés, sinon il faut une autre méthode. Une date au jour près : une ligne modifiée plus tard le même jour tombe entre deux extractions ; la parade est de relire le jour du filigrane (`>=` plutôt que `>`) et de compter sur l'idempotence.

Théorie : `Theorie_04_DE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. Chaque changement, avec son auteur, sa date, sa transaction, l'ancienne et la nouvelle valeur du champ. `KNA1` ne garde que l'état courant : le palier d'un concessionnaire avant la révision du 2024-07-01 n'y existe plus, alors que `CDPOS` le garde. La limite est que seuls les objets configurés sont suivis : chez Boréal, les fiches des concessionnaires, et rien d'autre.

Théorie : `Theorie_04_DE.md`, section 3.

8. Il voit les suppressions, et il ne coûte presque rien à la base, parce qu'il lit le journal des écritures plutôt que les tables. En retour, il demande l'accès à ce journal, un connecteur à exploiter, et un format d'événements à comprendre.

Théorie : `Theorie_04_DE.md`, section 3.

9. `numberMatched` est le nombre d'entités qui répondent à la requête, `numberReturned` le nombre d'entités de la page reçue. Il reste des pages tant que la réponse porte un lien `next`, et l'on contrôle à la fin que la somme des `numberReturned` égale `numberMatched`.

Théorie : `Theorie_04_DE.md`, section 4.

10. 400 : non, la requête est fausse et la même requête échouera encore. 429 : oui, après le délai demandé, que l'en-tête `Retry-After` indique parfois. 500 : oui, après un délai, c'est une panne du serveur. Une coupure du réseau sans code : oui, après un délai, comme une erreur 5xx.

Théorie : `Theorie_04_DE.md`, section 4.

11. Le délai d'attente double à chaque échec (2, 4, puis 8 secondes) : au lieu d'assaillir un serveur en difficulté, on lui laisse le temps de se remettre. Le nombre d'essais reste borné pour qu'une panne durable finisse par remonter comme une erreur, plutôt que de faire tourner le pipeline sans fin.

Théorie : `Theorie_04_DE.md`, section 4.

Questions pratiques

P1.

```sql
WITH etat AS (
    SELECT * FROM entrepot.raw.erp_vbak
    QUALIFY ROW_NUMBER() OVER (PARTITION BY MANDT, VBELN ORDER BY _lot DESC) = 1)
SELECT COUNT(*) FILTER (WHERE (ERDAT BETWEEN '20240701' AND '20240731')
                           OR (AEDAT BETWEEN '20240701' AND '20240731')) AS filigrane
FROM etat;

SELECT COUNT(*) AS lignes_du_lot FROM entrepot.raw.erp_vbak WHERE _lot = '2024-07';
```

Résultat : 588 des deux côtés. Dans l'état reconstruit, 577 commandes ont été créées en juillet 2024 et 58 y ont été modifiées ; la somme dépasse 588 parce que certaines sont dans les deux cas. L'extrait mensuel n'est rien d'autre que cette requête, lancée sur la base la nuit qui suit la fin du mois. La comparaison des deux dates en texte fonctionne ici parce que le format `AAAAMMJJ` se trie comme la date qu'il représente.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_04_DE.md`, section 3.

P2.

```sql
SELECT kunnr, name1, regio
FROM sap.kna1
WHERE loevm <> ''
ORDER BY kunnr;
```

Résultat : 4 concessionnaires fermés, `0000200001` Eastleigh-Vance Rec Vehicles (MB), `0000200044` Yarrowby-Dunn Motorsports (AB), `0000200057` Holloway-Brent Recreation (ON) et `0000200074` Sandhurst-Ivey Recreation (SK). `LOEVM` est l'indicateur de suppression logique de SAP : la fiche reste dans la table, marquée, ce qui permet justement à une capture par filigrane de voir la fermeture. L'état courant de `raw.erp_kna1`, reconstruit par lot, rend les quatre mêmes fiches.

Théorie : `Theorie_04_DE.md`, sections 2 et 3.

P3.

```sql
SELECT p.value_old, p.value_new, COUNT(*) AS concessionnaires
FROM sap.cdpos p
JOIN sap.cdhdr h USING (mandt, objectclas, objectid, changenr)
WHERE p.fname = 'KUKLA' AND h.udate = '20240701'
GROUP BY p.value_old, p.value_new
ORDER BY p.value_old, p.value_new;
```

Résultat : 4 lignes, 49 changements en tout. `01` vers `02` : 9 ; `02` vers `01` : 9 ; `02` vers `03` : 15 ; `03` vers `02` : 16. Le dictionnaire donne `01` pour Or, `02` pour Argent et `03` pour Bronze : 25 concessionnaires sont montés d'un palier (9 d'Argent à Or, 16 de Bronze à Argent), 24 sont descendus (9 d'Or à Argent, 15 d'Argent à Bronze), et aucun n'a sauté un palier. C'est ce que `KNA1` ne sait plus dire : la fiche ne porte que le palier d'aujourd'hui, et seuls les documents de modification gardent celui d'avant la révision.

Théorie : `Theorie_04_DE.md`, section 3.

P4. Avec des pages de 100, le script ne lit qu'une page, de 92 jours : les 92 jours annoncés y tiennent, et la réponse ne porte pas de lien `next`, ce qui arrête la boucle. Le fichier du bronze, lui, ne change pas d'un octet : son empreinte SHA-256 est la même qu'avec des pages de 50, et encore la même avec des pages de 10 000. La pagination change la façon de demander, jamais ce qu'on reçoit, parce que le script trie par date et écrit les jours dans l'ordre.

Demander toujours la plus grande page réduit le nombre de requêtes, pas le travail. Sur un long historique, une page énorme est une réponse lourde, longue à produire et à recevoir, et une coupure au milieu la fait perdre en entier, puisque la reprise redemande la page complète : une taille moyenne borne ce qu'un échec coûte. Et le serveur plafonne de toute façon : une demande de 15 000 observations horaires n'en rend que 10 000, et il faut suivre `next` quand même.

Théorie : `Theorie_04_DE.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

Diagnostics

E1. Le filigrane est posé sur la mauvaise date. `AEDAT` vaut `00000000` tant qu'une commande n'a jamais été modifiée : une commande créée hier et jamais retouchée n'entre dans aucune extraction. D'où un tableau de bord des nouvelles commandes toujours vide, alors que les rapports mensuels, qui lisent les extraits complets de l'ERP, tombent juste. Le correctif est de filtrer sur les deux dates, `ERDAT >= <filigrane> OR AEDAT >= <filigrane>`. Chez Boréal, en décembre 2025, cela fait 427 commandes au lieu de 37. À surveiller ensuite : le `OR` sur `ERDAT`, qui n'a pas d'index, fait lire la table entière ; sur une grosse table, il faut un index sur les deux colonnes, ou deux requêtes réunies.

Théorie : `Theorie_04_DE.md`, section 3.

E2. Deux fautes en une. `fetchall()` ramène les 40 millions de lignes en mémoire avant d'écrire quoi que ce soit : c'est l'erreur de mémoire sur le poste. Et la requête balaie la table entière pendant vingt minutes sur la base de production, ce qui consomme les disques et la mémoire dont l'ERP a besoin, et retient au passage les anciennes versions des lignes modifiées pendant la lecture. Le correctif : lire par paquets avec `fetchmany` et écrire au fil de l'eau ; ne demander que les colonnes utiles ; n'extraire que ce qui a changé, par filigrane, plutôt que la table entière ; et lancer le travail hors des heures de pointe, sur une réplique en lecture si elle existe, avec un compte qui n'a que le droit de lire.

Théorie : `Theorie_04_DE.md`, sections 2 et 3.

Questions à développer

A. Les commandes de l'ERP : un filigrane par requête, sur les deux dates (`ERDAT` et `AEDAT`), avec un index qui les couvre. La table est grosse, les modifications sont datées, et les suppressions n'existent pas : une commande annulée reçoit un motif de refus, elle ne disparaît pas. Les extraits mensuels sont déjà le produit de cette requête.

Les fiches des concessionnaires : les documents de modification, `CDHDR` et `CDPOS`. Un filigrane sur la fiche verrait qu'elle a changé, mais pas ce qui a changé : le palier d'avant la révision n'existe plus dans `KNA1`. Comme SAP tient déjà ce registre pour cet objet, il serait absurde de le refaire. La fermeture, elle, reste visible par `LOEVM`.

La table de stocks mise à jour toutes les minutes, avec suppressions : le journal de la base. Le filigrane ne voit pas les suppressions, et relire la table entière chaque minute est hors de question. Un outil comme Debezium rend chaque insertion, modification et suppression dans l'ordre, sans peser sur la base. S'il fallait s'en passer, faute d'accès au journal, on demanderait une suppression logique et une horodate de modification, et l'on accepterait de rater ce qui se passe entre deux lectures.

Théorie : `Theorie_04_DE.md`, section 3.

B. Ce que la proposition apporte est réel : l'état courant chaque matin plutôt qu'à la fin du mois, et un `STAGING` plus simple, puisqu'il n'y aurait plus de versions à départager. Ce qu'elle coûte : une lecture complète de l'ERP chaque nuit, qui grossit avec la base, une fenêtre nocturne à tenir, et une dépendance permanente à la disponibilité de l'ERP et du réseau, là où un fichier déposé attend tranquillement qu'on le ramasse. Ce qu'elle perd est le plus grave : l'historique. Les extraits gardent la mémoire de chaque état mensuel, y compris des données maîtres, et permettent de rejouer le passé et de prouver ce qui a été livré ; une extraction complète qui remplace n'a que l'état d'aujourd'hui, et une commande modifiée efface sa version précédente. La réponse défendable est donc de garder les

© 2026 iAlexMG. Tous droits réservés.

extraits comme source de vérité et d'ajouter, si la fraîcheur compte vraiment, une extraction incrémentale quotidienne par filigrane, qui s'ajoute à la `RAW` sans rien remplacer.

Théorie : `Theorie_04_DE.md`, sections 2 et 3.

Mises en situation

M1. Portail des permis de la Ville de Rivière-Blanche.

L'extraction initiale suit les curseurs : on demande une page de 500 permis, on garde le jeton `suivant` reçu, et l'on recommence jusqu'à ce qu'il n'y en ait plus. Les 900 000 permis font 1 800 requêtes, soit près de deux heures sous la limite de 1 000 requêtes par heure : on temporise pour rester sous le plafond plutôt que d'attendre les 429, et on respecte le délai demandé quand il en arrive un. Le jeton et le nombre de permis déjà reçus se gardent entre deux exécutions, dans un état du pipeline, pour qu'une reprise ne recommence pas au début.

L'extraction quotidienne demande les permis émis depuis le filigrane, la date d'émission la plus récente déjà reçue, en la relisant en entier (`>=`) et en comptant sur l'idempotence pour absorber les recoupements. Trois cents permis par jour tiennent en une requête. La preuve qu'il ne manque rien se fait par le compte : le total annoncé par l'API pour la période demandée doit égaler le nombre de permis chargés, et le journal des chargements en garde la trace. Un contrôle de continuité des numéros de permis, quand ils se suivent, ajoute une seconde preuve.

Théorie : `Theorie_04_DE.md`, sections 3 et 4.

M2. Assurances Val-Perdu.

Décision	**Ce que je propose**
Méthode de capture	un filigrane par requête, sur la date de modification, pour les polices ; les mouvements sont en ajout seul et se lisent par leur date ou leur identifiant croissant
Fréquence	chaque nuit pour les deux, dans la fenêtre de deux heures ; une réconciliation des clés une fois par semaine
Ce que je demande à l'équipe de l'ERP	une suppression logique sur les polices, ou une table d'audit des suppressions, et un index sur la date de modification
Les suppressions, en attendant	une réconciliation : extraire la seule liste des identifiants de polices, la comparer à l'état de la `RAW`, et marquer comme disparues celles qui n'y sont plus

Une extraction complète est hors de question : 400 Go ne passent pas dans une fenêtre de deux heures, et 2 milliards de lignes de mouvements encore moins. La table des mouvements est le cas facile, parce qu'une ligne y est écrite une fois et jamais modifiée : un filigrane sur l'identifiant croissant suffit, et l'on ne relit rien. Les polices sont le cas difficile, à cause des suppressions physiques et du refus d'accès au journal. La réconciliation des clés reste possible parce qu'elle ne lit

© 2026 iAlexMG. Tous droits réservés.

qu’une colonne, ce qui tient dans la fenêtre même sur des millions de lignes, mais elle ne dit pas quand la suppression a eu lieu, seulement qu’elle a eu lieu avant la lecture.

Ce qui me ferait changer d’avis : l’accès au journal des transactions, qui rendrait tout cela inutile et donnerait les suppressions à l’heure près ; une réplique de lecture, qui lèverait la contrainte de la fenêtre nocturne ; ou la découverte qu’un programme modifie des lignes sans toucher la date de modification, ce qui condamnerait le filigrane et obligerait à négocier autre chose.

Théorie : `Theorie_04_DE.md`, sections 2 et 3.

© 2026 iAlexMG. Tous droits réservés.